

04

The request lifecycle

What happens between an action in the interface and a change in the database

When a click creates, changes, or deletes something, a fixed sequence of steps runs, and each step is a place where behaviour can live, fail, or go missing. Knowing the stops on that path tells you where to look when something misbehaves, and where work belongs when it doesn't fit inside a request at all. This chapter names the path, the layers along it, and the machinery for work that outlives a click.

Assumes chapter 1's two sides.

The model

The path of one request

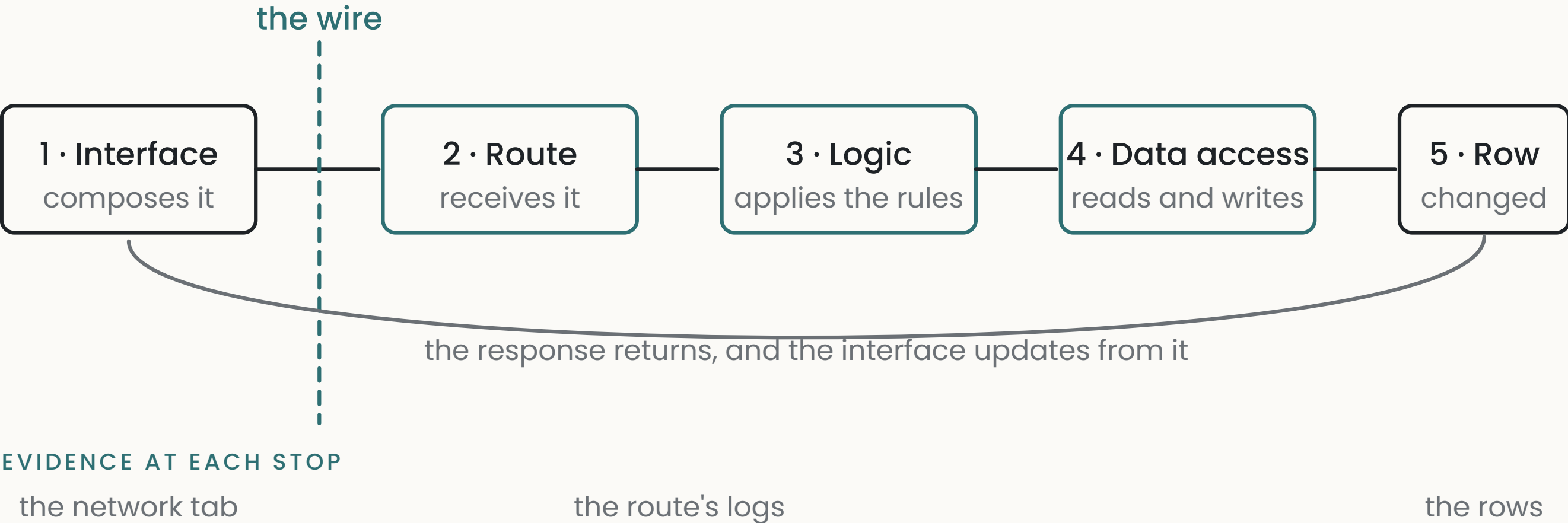
A write travels the same stops in every stack:

1. The **interface** composes a request from what the user did.
2. A **route** (or handler, or endpoint) on the server receives it — one route per kind of request the server answers.
3. **Logic** validates the input and applies the rules: is this allowed, what should change, what else follows from it.
4. **Data access** performs the reads and writes against the database.
5. A **response** returns, and the interface updates from it.

04

The request lifecycle

THE FIVE STOPS OF ONE WRITE



When something misbehaves, the question “which stop did this happen at” replaces “why is it broken” — and each stop has its own evidence: the request in the network tab, the route’s logs, the rows in the database.

Layers, and why crossing them costs

The server side of that path is conventionally kept in layers — routes, logic, data access — with each rule living in its layer. The cost of crossing shows up as duplication: when the interface writes straight to a table, or a business rule sits inside one route, the rule exists on one path and not on the others that touch the same data, including paths added later.

Chapter 6’s route-check limitation is one instance; the pattern is general. “Where should this rule live” has a conventional answer — with its layer — and a proposal that puts it elsewhere is worth a question.

Writes that go together

Some actions change several rows at once: an order and its line items, a transfer out of one account and into another, a membership plus the invitation it consumed. If the second write fails after the first succeeded, the data is half-changed, and nothing about it looks wrong from outside. A **transaction** is the database’s mechanism for this: the writes are grouped, and either all of them happen or none does. Generated code makes the choice silently in either direction — several writes in a row, or one grouped write — so “which writes in this feature must succeed together” is a question with a checkable answer.

APIs and resources

The set of requests a server answers is its **API**. The common convention organizes it around **resources** — the nouns of your product (events, RSVPs, members) — each supporting a standard verb set: list, fetch one, create, update, delete. The convention’s value to you is predictability: an assistant builds

and names endpoints along it, which means “list every endpoint, grouped by resource” produces an inventory whose gaps and stragglers you can judge without reading any code.

Work that outlives a request

A request is expected to answer within seconds; the user is waiting. Anything slow, scheduled, or worth retrying doesn’t fit inside one: sending emails, processing an upload, a nightly archive, syncing with another service. The machinery for it:

- A **background job** is work that runs with nobody waiting on it.
- A **queue** holds jobs until a worker picks them up, and re-queues the ones that fail — which is what makes retry possible.
- A **scheduler** starts work at a time rather than in response to anyone.

The request’s role then shrinks to: record what was asked for, put a job on the queue, answer. The email sends a moment later, and its failure becomes a retry instead of an error in the user’s face.

Webhooks

A webhook is the arrangement in reverse: you publish a URL, and another system sends you a request when something happens on their side — a payment settled, a message delivered. It arrives like any request from the outside, which means the chapter 1 questions apply to it: anyone can send a request to a public URL, so a webhook handler has to verify the sender. Chapter 9 covers that verification and the repeats these deliveries are known for.

What each endpoint costs, and who can call it

Every public endpoint can be called by anyone, as often as they like, with nothing but a script (chapter 1). For most endpoints that costs you nothing beyond load. Some are different: an endpoint that sends an email, calls a paid service, calls an AI model, or creates an account does something expensive or consequential per call, and a script calling it a thousand times a minute turns it into a spam relay, a bill, or a flood of fake rows. The guards are ordinary — require a login, cap how often one caller may hit it (a **rate limit**), put a ceiling on the spend — and each either exists or doesn't, which makes them enumerable.

What goes wrong

1 Slow work done inside the request

the user waits on something that didn't need them to.

ASK “For each endpoint, list everything it does before responding, and what the user experiences if each of those steps is slow or fails.”

CHECK make the slow step fail (a dead address does it) and watch what the user sees.

TELL *an email send, file processing, or a third-party call in the request path, with the response held until it finishes.*

3 Half a write

several rows that must change together, written one at a time with nothing grouping them.

ASK “For each endpoint that writes more than one table, say whether the writes are grouped in a transaction, and what the data looks like if the second write fails. Say ungrouped explicitly.”

CHECK make the second write fail in a test copy and read what the first one left behind.

TELL *two or more writes in sequence in one handler, with no transaction around them.*

2 Scheduled work on a host that won't run it

a job that works locally and never runs deployed.

ASK “List every task that runs on a schedule or for longer than ten seconds, and say what triggers each one in production. Say nothing where nothing does.”

CHECK trigger it in the deployed environment and find the evidence it ran.

TELL *a background job or long-running task on a platform with execution limits (chapter 5 explains why those exist).*

04

The request lifecycle

4 The unmetered endpoint

a route that does something expensive per call, open to anyone at any rate.

ASK “List every endpoint that sends a message, calls a paid service, calls an AI model, or creates an account. For each: who can call it, how often one caller may call it, and what a thousand calls in a minute would cost or produce. Write anyone and unlimited where those are the answers.”

CHECK call one of them fifty times in a row from the console and see what stops you.

TELL *an endpoint that sends, spends, or creates, with no login requirement and no rate limit.*

THE DIRECT TEST

Ask for the endpoint inventory: every request the server answers, grouped by resource, with what each reads and writes. The list is judgeable the way a schema is — resources you don’t recognize, verbs missing from a resource that should have them, and two endpoints writing the same table are all visible without reading code. It’s also the map the chapter 6 asks run over.

Going deeper

These prompts are for your assistant, and each comes with a note on what a good response looks like.

D1 · Trace one action

GROUNDING EXPLANATION

Take one action in my app: [name it — e.g. RSVP to an event]. Walk me through everything that happens from the click to the database and back, naming each stop: the request sent, the route that receives it, the rules applied, the tables touched, the response, and what updates on screen. Point at the actual file for each stop.

WHAT A GOOD RESPONSE LOOKS LIKE

A good response is the five-stop path with your filenames on it. A stop it can't point to a file for is worth a follow-up — the rule you assumed lives somewhere may live nowhere.

D3 · Walk the job failure

WALK THE FAILURE PATH

Walk me through what happens when [a background job — the reminder email, the nightly archive] fails halfway. Step by step: what's been done, what hasn't, whether it retries, what evidence the failure leaves, and who or what would ever notice.

WHAT A GOOD RESPONSE LOOKS LIKE

A good response ends with a named place the failure becomes visible. "Nothing would notice" is a finding, and a common one — background failures have nobody waiting on them by design.

D2 · The endpoint inventory

AUDIT

List every request my server answers — every route and method — grouped by resource. For each: what it reads, what it writes, whether anything else writes the same data, and who can call it — anyone, any logged-in user, or a specific role. Full table, no summarizing.

WHAT A GOOD RESPONSE LOOKS LIKE

A good response is the inventory described above. Two columns carry the findings: two writers to one table is where layer-crossing shows up, and "anyone" next to an endpoint that sends or spends is the unmetered-endpoint entry.

04

The request lifecycle

D4 · The path quiz

PREDICTION QUIZ

Quiz me on my app’s request path. One at a time, name a failure at a specific stop — the request never arrives, the validation rejects, the database write fails, the response is lost — and ask me what the user sees and what state the data is in. Wait for my answer, then correct me with the reason.

WHAT A GOOD RESPONSE LOOKS LIKE

Commit before each correction. The response-is-lost case is the one most people get wrong: the write happened, and only the confirmation didn’t.

WHERE THIS CONNECTS

Chapter 1 · Client and server owns the wire between stops one and two. **Chapter 3 · Source of truth** covers the copies made along this path. **Chapter 5 · Production** explains the platform limits behind the scheduled-work entry, and **chapter 9 · Third-party integrations** picks up webhooks and the calls that leave your app — including the bill an unmetered endpoint runs up there. **Chapter 8 · Injection** covers the text a request carries into a query or a prompt.